

Praca Dyplomowa Inżynierska

Stanisław Stankiewicz
217442

Modularny system zadań logicznych zrealizowany w języku C++ dla mikrokontrolera ATmega328P

Modular logic task system implemented in C++ language for the
ATmega328P microcontroller

Praca dyplomowa na kierunku:
Informatyka

Praca wykonana pod kierunkiem
dra inż. Artura Wilińskiego
Instytut Informatyki Technicznej
Katedra Systemów Informatycznych

Warszawa, rok 2026



SZKOŁA GŁÓWNA
GOSPODARSTWA
WIEJSKIEGO

Wydział Zastosowań
Informatyki
i Matematyki

Oświadczenie Promotora pracy

Oświadczam, że niniejsza praca została przygotowana pod moim kierunkiem i stwierdzam, że spełnia ona warunki do przedstawienia tej pracy w postępowaniu o nadanie tytułu zawodowego.

Data

Podpis promotora

Oświadczenie autora pracy

Świadom/a odpowiedzialności prawnej, w tym odpowiedzialności karnej za złożenie fałszywego oświadczenia, oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami prawa, w szczególności z ustawą z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (Dz. U. 2019 poz. 1231 z późn. zm.)

Oświadczam, że przedstawiona praca nie była wcześniej podstawą żadnej procedury związanej z nadaniem dyplomu lub uzyskaniem tytułu zawodowego.

Oświadczam, że niniejsza wersja pracy jest identyczna z załączoną wersją elektroniczną. Przyjmuję do wiadomości, że praca dyplomowa poddana zostanie procedurze antyplagiatowej.

Data

Podpis autora pracy

Streszczenie

Tematem pracy jest zaprojektowanie i implementacja modułowego systemu interaktywnych zadań logicznych opartego na mikrokontrolerach ATmega328P. Projekt zakładał stworzenie systemu wbudowanego stymulującego współpracę grupy użytkowników rozwiązujących powiązane zagadki pod presją czasu. Architektura umożliwia dowolną konfigurację oraz łatwe rozszerzanie układu o kolejne niezależne moduły. Praca obejmuje fazę koncepcyjną, projektowanie sprzętowe, implementację protokołu komunikacyjnego I²C oraz integrację oprogramowania w języku C++. Wyniki testów wskazują, że zaimplementowane podsystemy działają stabilnie, a przyjęta sieć Nadrzędny-Podrzędny (ang. *Master-Slave*) zapewnia skalowalność środowiska.

modułowy system wbudowany, C++, I2C, ATmega328P, praca inżynierska, SGGW

Słowa kluczowe – Modular logic task system implemented in C++ language for the ATmega328P microcontroller

Summary

The subject of this thesis is the design and implementation of a modular system of interactive logic tasks based on ATmega328P microcontrollers. The project involved creating an embedded system that stimulates the cooperation of a group of users solving interrelated puzzles under time pressure. The system architecture was planned to allow for arbitrary configuration and easy expansion with additional independent modules. The thesis covers the conceptual phase, hardware design, implementation of a communication protocol based on the I²C bus, and software integration in C++. Validation test results indicate that the implemented subsystems exhibit stable operation, and the adopted Master-Slave network ensures scalability of the environment.

embedded system, C++, I2C, ATmega328P, engineering thesis, WULS

Keywords –

Spis treści

1	Wstęp	9
1.1	Cel pracy	9
1.2	Analiza istniejących rozwiązań	10
1.3	Zakres pracy	10
2	Narzędzia i użyte technologie	12
2.1	Mikrokontroler ATmega328P	12
2.2	Środowisko Arduino IDE	13
2.3	Biblioteka komunikacji I ² C – Wire.h	13
2.4	Biblioteki graficzne Adafruit	14
2.5	Biblioteka wyświetlacza LiquidCrystal	14
2.6	Biblioteka TM1637Display	15
2.7	Biblioteka interfejsu SPI	15
3	Projekt systemu zadań logicznych	17
3.1	Specyfikacja wymagań	17
3.1.1	Wymagania funkcjonalne	17
3.1.2	Wymagania нефункционалне	18
3.2	Architektura systemu	20
3.3	Konstrukcja i montaż modułów	20
3.4	Diagramy klas	21
3.4.1	Diagram klas biblioteki ModuleComms	21
3.4.2	Diagram klas modułów zagadek	23
3.4.3	Diagram klas warstwy sprzętowej i bibliotek zewnętrznych	24
4	Implementacja systemu	26
4.1	Architektura oprogramowania	26
4.2	Autorska biblioteka komunikacyjna ModuleComms	26
4.3	Wielozadaniowość kooperatywna i wstrzymania nieblokujące	30

4.4	Implementacja obwodów i modułów wbudowanych	30
4.4.1	Adresacja i topologia modułów	30
4.4.2	Moduł nadrzędny	31
4.4.3	Moduł sekwencyjny	35
4.4.4	Moduł arytmetyczny	37
4.4.5	Moduł z przełącznikiem	39
4.4.6	Moduł labiryntu	41
4.4.7	Moduł radia	44
5	Testy i działanie systemu	47
5.1	Analiza wykorzystania zasobów sprzętowych	47
5.2	Weryfikacja systemu poprzez testy i pomiary	48
5.2.1	Wyniki testów sprzętowych	49
5.2.2	Analiza wydajności i częstotliwości komunikacji	50
6	Podsumowanie i wnioski końcowe	52
	Bibliografia	54
	Spis rysunków	56
	Spis tabel	57

1 Wstęp

Wraz z dynamicznym rozwojem systemów wbudowanych (ang. *embedded systems*) oraz postępującą miniaturyzacją sprzętu elektronicznego mikrokontrolery stały się powszechnie dostępne dla szerokiego grona inżynierów i konstruktorów. Umożliwiają one projektowanie zaawansowanych, interaktywnych urządzeń fizycznych, które wymagają od użytkownika logicznego myślenia oraz szybkiego podejmowania decyzji. Istniejące na rynku rozwiązania komercyjne (wykorzystywane m.in. w pokojach zagadek typu *escape room* lub w celach edukacyjno-szkoleniowych) rzadko jednak oferują otwartą i modułową architekturę opartą na tanich, 8-bitowych mikrokontrolerach tworzących skalowalną sieć. Z tego powodu podjęto próbę konstrukcji autorskiej platformy.

1.1 Cel pracy

Głównym celem niniejszej pracy jest zaprojektowanie architektoniczne oraz wdrożenie modułarnego systemu zadań logicznych. System wymusza równoległe rozwiązywanie różnorodnych procedur zadaniowych przez wielu uczestników w wyznaczonym czasie. Rozwiązanie opiera się na zdecentralizowanych interfejsach wejściowych oraz centralnym nadzorze jednostki zarządzającej.

Temat wybrano w celu weryfikacji zagadnień związanych z programowaniem 8-bitowych mikrokontrolerów rodziny AVR, w szczególności układu ATmega328P [1], integracją układów elektronicznych oraz implementacją niezawodnej komunikacji między modułami. Konstrukcja systemu czasu rzeczywistego (ang. *real-time system*) stanowi złożone zagadnienie techniczne. Wymagała ona m.in. zbudowania warstwy transportowej opartej na magistrali I²C w celu zapewnienia bezkolizyjnej wymiany danych pomiędzy jednostką nadrzędną, zarządzającą logiką gry, a niezależnymi węzłami wykonawczymi. Ze względu na specyfikę projektu konieczna była optymalizacja kodu C++ pod kątem ograniczonej pamięci SRAM, której pojemność w układach ATmega328P wynosi 2 KB.

1.2 Analiza istniejących rozwiązań

Projektowany system czerpie inspirację z popularnej gry kooperacyjnej *Keep Talking and Nobody Explodes*, w której gracze muszą wspólnie rozbroić ładunek, komunikując się i rozwiązując zagadki logiczne. Na rynku istnieją również komercyjne platformy dla pokoi zagadek (np. *Escape Room Master*), oferujące gotowe sterowniki i oprogramowanie. Jednak większość tych rozwiązań opiera się na drogich sterownikach PLC lub mikrokomputerach typu Raspberry Pi. Niniejsza praca udowadnia, że podobną funkcjonalność można osiągnąć przy użyciu znacznie tańszych i bardziej energooszczędnych 8-bitowych mikrokontrolerów, zachowując przy tym pełną modularność i łatwość rozbudowy.

Dodatkową wartością zaproponowanego rozwiązania jest połączenie kilku mechanizmów rzadko występujących łącznie w prostych systemach AVR: automatycznego wykrywania modułów na magistrali I²C, globalnej deterministycznej konfiguracji scenariusza gry (*seed*), propagacji liczby błędów do wszystkich węzłów oraz odpornej na przypadkowe wyzwolenie procedury restartu.

1.3 Zakres pracy

Niniejsza praca inżynierska obejmuje pełen cykl projektowy i programistyczny – od zaplanowania architektury systemu i opracowania algorytmiki oprogramowania aż do ostatecznych testów stabilności. Wyszczególniony wykaz obszarów zadaniowych zrealizowanych w ramach niniejszej pracy obejmuje:

- opracowanie koncepcji sprzętowej i programowej topologii rozproszonej typu *Master-Slave*, wykorzystującej mikrokontrolery ATmega328P [1],
- wdrożenie biblioteki `ModuleComms` opartej na protokole I²C, obsługującej automatyczne wykrywanie modułów na magistrali (tzw. skanowanie adresów),
- zaprogramowanie stacji *Master*, odpowiedzialnej za odliczanie czasu przy użyciu wielozadaniowości asynchronicznej oraz odbieranie informacji o stanie zadań z modułów,
- zaprogramowanie w języku C++ modułów: arytmetycznego, labiryntu (wyświetlacz OLED), radiowego (dekodowanie alfabetu Morse’a), sekwencyjnego oraz modułu z przełącznikiem dźwigniowym,
- optymalizację użycia pamięci SRAM poprzez wykorzystanie pamięci Flash

(dyrektywa **PROGMEM**) do przechowywania danych statycznych,

- realizację testów z uwzględnieniem sprzętowej i programowej eliminacji drgań styków (ang. *debouncing*).

Tak zdefiniowany zakres porządkuje dalszą strukturę pracy: w kolejnym rozdziale przedstawiono technologie i biblioteki wykorzystane w projekcie, następnie omówiono założenia projektowe, implementację oraz wyniki testów.

2 Narzędzia i użyte technologie

W niniejszym rozdziale przedstawiono technologie, narzędzia oraz biblioteki wykorzystane w niniejszej pracy. Dla każdego komponentu opisano jego rolę oraz sposób wykorzystania. Realizacja pracy opierała się na sprawdzonych rozwiązaniach otwartoźródłowych (ang. *open-source*), co pozwoliło skupić się na logice gry zamiast na tworzeniu sterowników od podstaw.

2.1 Mikrokontroler ATmega328P

ATmega328P [1] to 8-bitowy mikrokontroler z rodziny AVR, posiadający 32 KB pamięci programu (Flash), 2 KB pamięci operacyjnej (SRAM) oraz 1 KB pamięci danych (EEPROM). Dzięki architekturze RISC większość instrukcji wykonuje w jednym cyklu zegara, co zapewnia dobrą wydajność przy niskim poborze energii.

Wybór tego układu podyktowany był trzema czynnikami. Po pierwsze, stanowi on fundament popularnej platformy Arduino Uno, co daje dostęp do bardzo dużej liczby bibliotek oraz materiałów edukacyjnych. Po drugie, oferuje natywną obsługę peryferiów pracujących w standardzie 5 V, co eliminuje potrzebę stosowania konwerterów napięć. Po trzecie, posiada sprzętowy kontroler I²C (TWI), który jest fundamentem komunikacji w tym systemie.

W niniejszej pracy układy pracują pod kontrolą wewnętrznego oscylatora RC skonfigurowanego na częstotliwość 8 MHz. Wykorzystanie wbudowanego źródła sygnału zegarowego, zamiast zewnętrznego rezonatora kwarcowego dostarczającego sygnał taktujący, obniża złożoność i koszt obwodu, a jednocześnie pozwala na pracę w poszerzonym zakresie napięć zasilania od 2.7 V do 5.5 V. Choć wbudowany oscylator charakteryzuje się tolerancją wahań do $\pm 10\%$ (w stałych warunkach pokojowych błąd oscyluje w granicach $\pm 1\text{--}2\%$), odchylenia te są pomijalne w kontekście działania gry oraz obsługi magistrali I²C. Konfigurację niskopoziomową mikrokontrolera (ustawienie bitów konfiguracyjnych (ang. *fuse bits*), odpowiedzialnych za trwałe ustawienia sprzętowe układu) przeprowadzono bezpośrednio przy użyciu narzędzia wiersza poleceń `avrdude`, co umożliwiło precyzyjne sterowanie parametrami pracy układu bez użycia dodatkowych nakładek systemowych. Bezpośred-

nie wgrywanie oprogramowania przez złącze programowania wewnątrzsistemowego (ang. *In-System Programming* — ISP) pozwoliło całkowicie wyeliminować program rozruchowy (ang. *bootloader*), dzięki czemu kod programu ma do dyspozycji pełne 32 KB pamięci Flash. Jest to istotna optymalizacja względem standardowej platformy Arduino Uno, w której część pamięci programu stale zajmuje wspomniany program rozruchowy.

2.2 Środowisko Arduino IDE

Arduino IDE [2] to środowisko programistyczne ułatwiające tworzenie oprogramowania na mikrokontrolery. Pozwala na pisanie kodu w C++ z wykorzystaniem gotowych bibliotek obsługujących m.in. przerwania czy komunikację.

Wybór Arduino IDE, zamiast alternatyw takich jak PlatformIO czy Atmel Studio, podyktowany był przede wszystkim dojrzałością ekosystemu bibliotek kompatybilnych z platformą AVR oraz niskim progiem wejścia przy prototypowaniu. Choć PlatformIO oferuje bardziej zaawansowane zarządzanie zależnościami, Arduino IDE zapewniło wystarczającą funkcjonalność przy jednoczesnej prostocie konfiguracji wielomodułowego projektu.

W ramach niniejszej pracy proces programowania układów realizowano za pośrednictwem wspomnianego wcześniej narzędzia `avrdude` oraz zewnętrznego programatora sprzętowego USBasp. Na potrzeby seryjnego przygotowania mikrokontrolerów opracowano dedykowany adapter wyposażony w gniazdo typu ZIF (ang. *Zero Insertion Force*) z mechanizmem dźwigniowym. Rozwiązanie to umożliwiło szybką i bezpieczną wymianę układów brzegowych bez ryzyka uszkodzenia ich wyprowadzeń, zapewniając stabilne połączenie z programatorem.

2.3 Biblioteka komunikacji I²C — Wire.h

Wire.h [4] jest standardową biblioteką środowiska Arduino, realizującą obsługę protokołu I²C (*Inter-Integrated Circuit*) za pośrednictwem wbudowanego sprzętowego kontrolera TWI mikrokontrolera ATmega328P. Biblioteka przejmuje zarządzanie rejstrami taktowania przesyłu, przerwaniami oraz identyfikacją sprzętową sygnałów potwierdzających komunikację (ACK/NACK).

Protokół I²C został wybrany jako magistrala komunikacyjna systemu ze względu na minimalizację liczby wymaganych połączeń sygnałowych — do połączenia do-

wolnej liczby urządzeń wystarczają zaledwie dwie linie sygnałowe (SDA i SCL) oraz wspólne zasilanie. W porównaniu z alternatywami, takimi jak SPI (wymagający dodatkowej linii CS dla każdego urządzenia) czy UART, I²C stanowi optymalny kompromis między prostotą połączeń a skalowalnością. Co istotne, protokół UART jest interfejsem asynchronicznym, co przy braku zewnętrznego oscylatora kwarcowego i pracy na wewnętrznym oscylatorze RC mogłoby prowadzić do błędów synchronizacji ramek przy wahaniach temperatury. Magistrala I²C, jako interfejs synchroniczny (wykorzystujący linię zegarową SCL), jest niewrażliwa na niedokładności taktowania mikrokontrolerów.

W niniejszej pracy biblioteka `Wire.h` stanowi fundament autorskiej biblioteki `ModuleComms`, realizując fizyczną warstwę transportową komunikacji między jednostką Master a modułami Slave.

2.4 Biblioteki graficzne Adafruit

Rodzina bibliotek Adafruit obejmuje `Adafruit_GFX` [6] — uniwersalny rdzeń graficzny 2D — oraz `Adafruit_SH110X` [5] — dedykowany sterownik dla wyświetlaczy OLED opartych na układzie SH1106G.

Biblioteki te wybrano ze względu na ich szeroką kompatybilność z ekosystemem Arduino, aktywne wsparcie społeczności oraz zoptymalizowane algorytmy rysowania, dostosowane do ograniczeń pamięciowych mikrokontrolerów 8-bitowych. Alternatywna biblioteka `U8g2lib`, choć oferuje podobną funkcjonalność, wykazuje większe zapotrzebowanie na pamięć SRAM, co stanowi istotne ograniczenie na platformie ATmega328P dysponującej zaledwie 2 KB tego zasobu.

W niniejszej pracy biblioteki Adafruit wykorzystano w modułach labiryntu oraz radiowym do obsługi monochromatycznych wyświetlaczy OLED o rozdzielczości 128x64 pikseli. Warstwa `Adafruit_GFX` umożliwia rysowanie prymitywów graficznych (prostokątów, okręgów, glifów tekstowych) oraz renderowanie bitmap przechowywanych w pamięci Flash za pośrednictwem dyrektywy `PROGMEM`, co pozwala na obsługę dużych zasobów graficznych bez obciążania ograniczonej pamięci SRAM.

2.5 Biblioteka wyświetlacza LiquidCrystal

Biblioteka `LiquidCrystal` [3] jest standardowym narzędziem środowiska Arduino służącym do obsługi wyświetlaczy alfanumerycznych opartych na sterowniku Hita-

chi HD44780. Pozwala ona na komunikację z panelem LCD w trybie 4-bitowym lub 8-bitowym, co umożliwia elastyczne zarządzanie zasobami sprzętowymi mikrokontrolera.

W niniejszej pracy wykorzystano tryb 4-bitowy, który wymaga podłączenia sześciu linii sygnałowych (RS, EN oraz cztery linie danych). Choć rozwiązanie to zajmuje więcej pinów mikrokontrolera niż alternatywne interfejsy szeregowy, zapewnia ono najwyższą wydajność odświeżania oraz pełną kompatybilność z podstawowymi bibliotekami bez konieczności stosowania dodatkowych układów pośredniczących, takich jak ekspandery I²C. Wybór ten był możliwy dzięki rezygnacji z innych peryferiów w module arytmetycznym, co pozostawiło wystarczającą liczbę wolnych wyprowadzeń ATmega328P. W ramach niniejszej pracy biblioteka służy do wyświetlania treści zadań matematycznych oraz wprowadzanych przez użytkownika odpowiedzi na panelu LCD 16x2.

2.6 Biblioteka TM1637Display

TM1637Display [7] jest biblioteką obsługującą sprzętowy sterownik TM1637, stosowany w poczwórnych wyświetlaczach 7-segmentowych. Sterownik ten wykorzystuje dwuprzewodowy protokół komunikacyjny zbliżony koncepcyjnie do I²C, lecz wymagający dedykowanych linii CLK i DIO.

Decyzja o zastosowaniu wyświetlacza TM1637 w jednostce nadrzędnej, zamiast np. wyświetlacza OLED, podyktowana była niskim kosztem komponentu, doskonałą czytelnością dużych cyfr z dalszej odległości oraz minimalnym zapotrzebowaniem na zasoby mikrokontrolera. Biblioteka realizuje transmisję danych bez stosowania blokujących opóźnień typu `delay()`, co jest kluczowe dla zachowania ciągłości asynchronicznego nasłuchu na magistrali I²C w pętli głównej programu.

W niniejszej pracy wyświetlacz TM1637 pełni rolę głównego wskaźnika upływającego czasu gry w formacie MM:SS, informując uczestników o zbliżającym się jego końcu.

2.7 Biblioteka interfejsu SPI

SPI.h jest standardową biblioteką środowiska Arduino, realizującą obsługę protokołu *Serial Peripheral Interface* (SPI). Interfejs ten umożliwia szybką, synchroniczną komunikację szeregową z peryferiami za pośrednictwem czterech linii sygnałowych:

MOSI (*Master Out Slave In*), MISO (*Master In Slave Out*), SCK (*Serial Clock*) oraz CS (*Chip Select*).

Protokół SPI został zastosowany w module labiryntu zamiast I²C z dwóch kluczowych powodów. Po pierwsze, magistrala SPI oferuje znacząco wyższą przepustowość, niezbędną do wydajnego przesyłania pełnych buforów ramki obrazu (1 KB) do wyświetlacza OLED bez blokowania pętli głównej. Po drugie, mikrokontrolery ATmega328P posiadają tylko jeden sprzętowy interfejs I²C, który w tym module jest już wykorzystywany do pracy w trybie Slave na potrzeby komunikacji systemowej. Jednoczesna obsługa wyświetlacza jako Master na tej samej magistrali byłaby programowo i sprzętowo nieefektywna, wymagając ciągłego i ryzykownego przełączania trybów pracy kontrolera TWI.

3 Projekt systemu zadań logicznych

W niniejszym rozdziale przedstawiono projekt modularnego systemu zadań logicznych. Głównym celem projektu było opracowanie skalowalnej i łatwej w rozszerzaniu platformy sprzętowo-programowej, umożliwiającej jednoczesną, interaktywną pracę wielu uczestników z niezależnymi modułami zagadek, koordynowanymi przez centralną jednostkę zarządzającą. W kolejnych podrozdziałach szczegółowo opisano specyfikację wymagań, architekturę systemu oraz strukturę klas obiektowych.

3.1 Specyfikacja wymagań

Kluczowym etapem projektu jest określenie wymagań, które system musi spełniać. Na podstawie analizy problemu, przeglądu podobnych rozwiązań (np. w pokojach zagadek typu *escape room*) oraz ograniczeń mikrokontrolerów AVR, wyodrębniono wymagania funkcjonalne i нефункционалне.

3.1.1 Wymagania funkcjonalne

Wymagania funkcjonalne definiują zakres operacji i możliwości, które system musi dostarczyć użytkownikowi końcowemu. Kluczowe funkcje realizowane przez projektowany system zostały zestawione w Tabeli 3.1.

Tabela 3.1. Wymagania funkcjonalne systemu

Numer	Nazwa	Opis
WF-01	Koordinacja czasu rozgrywki	Centralny węzeł Master zarządza odliczaniem czasu dla wszystkich modułów. Błędy popełniane przez graczy powodują przyspieszenie odliczania czasu.
WF-02	Niezależna logika modułów zagadek	Każdy moduł posiada własny algorytm rozwiązywania, a komunikacja z jednostką nadrzędną odbywa się wyłącznie na żądanie węzła głównego.
Kontynuacja na następnej stronie		

Tabela 3.1 – kontynuacja z poprzedniej strony

Numer	Nazwa	Opis
WF-03	Determinizm konfiguracji (Seed)	System inicjuje przed startem dystrybucję globalnego numeru konfiguracji po łączu I ² C, determinując pseudolosowe zdarzenia we wszystkich układach.
WF-04	Obsługa wielu typów modułów	System obsługuje co najmniej pięć typów modułów: arytmetyczny, labiryntu, radiowy, sekwencyjny oraz z przełącznikiem.
WF-05	Detekcja i propagacja błędów	Niepoprawne działanie użytkownika generuje zdarzenie błędu rejestrowane globalnie przez Master. Akumulacja błędów wpływa na zachowanie pozostałych modułów.
WF-06	Automatyczne wykrywanie topologii	Serwer Master po uruchomieniu skanuje adresy magistrali I ² C w celu automatycznej identyfikacji podłączonych modułów bez ręcznej konfiguracji.
WF-07	Sygnalizacja dźwiękowa i wizualna	System zapewnia eskalujące ostrzeżenia dźwiękowe oraz wizualne informujące o zbliżającym się końcu limitu czasowego oraz o stanie błędu.
WF-08	Mechanizm restartu z zabezpieczeniem	Restart gry wymaga trzykrotnej aktywacji przełącznika w ustalonym oknie czasowym, zapobiegając przypadkowemu zresetowaniu rozgrywki.

Zdefiniowane wymagania kładą nacisk na samodzielność modułów przy centralnej koordynacji. Obejmują one mechanizmy zarządzania rozgrywką (odliczanie czasu, błędy) oraz funkcje systemowe, takie jak automatyczne wykrywanie modułów czy konfiguracja gry za pomocą ziarna (ang. *seed*).

3.1.2 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne definiują cechy jakościowe systemu oraz techniczne ograniczenia projektu. Kluczowe wymagania z tej kategorii zestawiono w Tabeli 3.2.

Tabela 3.2. Wymagania niefunkcjonalne systemu

Numer	Nazwa	Opis
WN-01	Modularność sprzętowa (Plug-and-Play)	Stacje podłączane są do wspólnej szyny I ² C na współdzielonym zasilaniu (+5 V i GND), co ogranicza liczbę połączeń montażowych i zwiększa odporność na chwilowe przerwy w łączności.
WN-02	Wielozadaniowość nieblokująca	Rezygnacja z wywołań <code>delay()</code> w logice rdzeniowej. Ocena stanów opiera się na obliczaniu różnicy czasu z wykorzystaniem systemowego stopera <code>millis()</code> .
WN-03	Efektywne wykorzystanie pamięci	System funkcjonuje w ramach limitów ATmega328P (2 KB SRAM, 32 KB Flash). Duże struktury danych alokowane są w pamięci Flash za pośrednictwem <code>PROGMEM</code> .
WN-04	Skalowalność	Architektura umożliwia rozszerzenie o dodatkowe moduły bez modyfikacji oprogramowania Master (do 126 urządzeń na magistrali I ² C).
WN-05	Niezawodność komunikacji	Protokół transportowy jest odporny na chwilowe zaniki łączności z pojedynczymi modułami bez destabilizacji pracy pozostałych węzłów.
WN-06	Koszt i dostępność komponentów	Projekt bazuje na tanich, powszechnie dostępnych komponentach elektronicznych w celu ograniczenia kosztów produkcji prototypu.
WN-07	Stabilność zasilania	System funkcjonuje w zakresie napięć 2.7–5.5 V, wykorzystując wewnętrzny oscylator RC 8 MHz z tolerancją wahań $\pm 1\text{--}2\%$.

Wymagania te stawiają priorytet na wielozadaniowość oraz optymalizację pamięci — co jest krytyczne przy zaledwie 2 KB pamięci SRAM. Istotna jest także modularność typu Plug-and-Play, pozwalająca na dowolne podłączanie modułów bez zmian w oprogramowaniu sterownika głównego.

3.2 Architektura systemu

System oparto na architekturze nadrzędno-podrzędnej (ang. *Master-Slave*), gdzie centralna jednostka koordynuje pracę modułów przez magistralę I²C (*Inter-Integrated Circuit*). Jest to szeregowy interfejs komunikacyjny wykorzystujący dwie linie sygnałowe: SDA (*Serial Data*) do przesyłu danych oraz SCL (*Serial Clock*) do synchronizacji zegarowej. Wybór tej magistrali podyktowany był jej oszczędnością w liczbie wymaganych połączeń oraz natywnym wsparciem przez sprzętowy kontroler TWI (*Two Wire Interface*) mikrokontrolera ATmega328P [1]. Zastosowanie języka C++ pozwoliło na programowanie obiektowe z pełną kontrolą nad alokacją pamięci.

Biblioteka `ModuleComms` zarządza komunikacją na niskim poziomie, wykorzystując sprzętowy protokół I²C. Master inicjuje zapytania i przesyła informacje o czasie czy błędach, natomiast klasa `Slave` automatycznie obsługuje te żądania po stronie modułów. Dzięki zastosowanej abstrakcji programista modułu nie musi zajmować się obsługą magistrali, co pozwala skupić się na samej logice zagadki. Wyróżnia się trzy główne fazy pracy systemu:

1. **Faza wykrywania (ang. *Discovery*):** Wywołanie pętli wysyłkowej z komendą 0x07 (`CMD_IDENTIFY`), skanującej adresy od 1 do 126 z zapytaniem o rzetelną odpowiedź ACK sprzętu.
2. **Faza propagacji i rejestracji startu (ang. *Propagation and Start*):** Przeprowadzenie inicjacyjnej transakcji rozgłaszanej z flagą `CMD_START_GAME`.
3. **Faza testowania cyklicznego stanu (ang. *Run Loop*):** Ciągłe odpytywanie modułów o ich stan logiczny, przechwytywanie zdarzeń błędu i sukcesu.

3.3 Konstrukcja i montaż modułów

Ważnym aspektem projektu była integracja komponentów elektronicznych w działające moduły. Każda stacja została przygotowana w sposób umożliwiający testowanie i weryfikację założeń projektowych. Wszystkie połączenia krytyczne (zasilanie, magistrala I²C) zostały wykonane w sposób zapewniający stabilność komunikacji.

W procesie realizacji skupiono się na:

- **Przygotowaniu stanowisk:** Przyciski, enkoder oraz wyświetlacze zostały połączone z mikrokontrolerami zgodnie z opracowanymi schematami.
- **Zasilaniu:** Całość systemu zasilana jest napięciem 5 V ze wspólnego źródła,

co zapewnia kompatybilność z mikrokontrolerami ATmega328P.

- **Integracji peryferiów:** Każdy moduł posiada dedykowany zestaw elementów (np. matryce LED, ekrany LCD/OLED) ściśle współpracujących z warstwą oprogramowania.

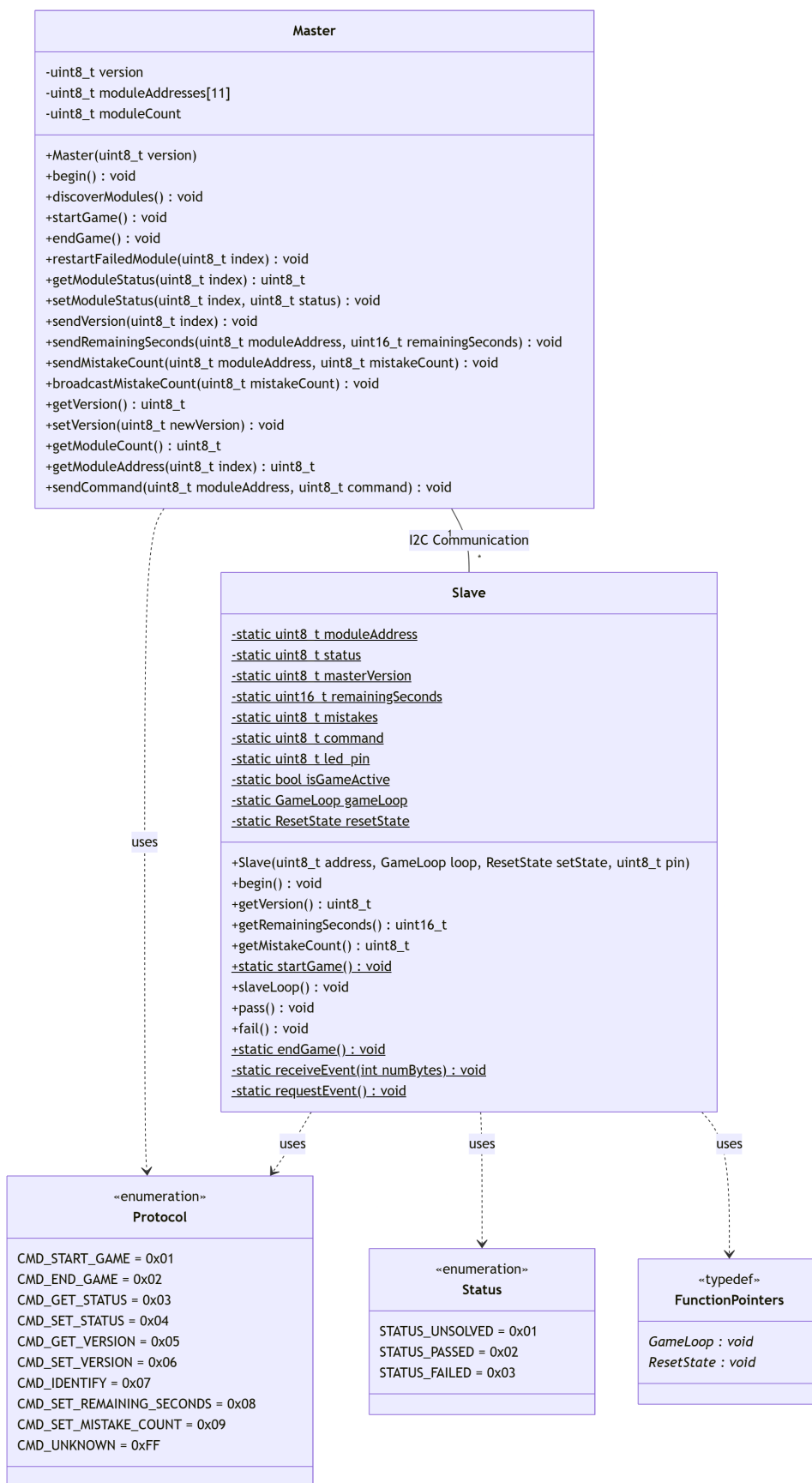
3.4 Diagramy klas

W niniejszej sekcji przedstawiono diagramy klas dokumentujące strukturę obiektową zaprojektowanego systemu. Diagramy opracowano zgodnie z notacją UML 2.0.

3.4.1 Diagram klas biblioteki ModuleComms

W celu sformalizowania struktury oprogramowania oraz ułatwienia procesu implementacji, posłużono się notacją UML (*Unified Modeling Language*). Diagram klas jest diagramem strukturalnym, przedstawiającym system za pomocą wizualizacji atrybutów, metod oraz relacji pomiędzy klasami. Jest on powszechnie stosowanym narzędziem w projektowaniu systemów opartych na programowaniu obiektowym.

Centralnym elementem architektury komunikacyjnej jest autorska biblioteka **ModuleComms**, której strukturę obiektową przedstawiono na Rysunku 3.1. Diagram prezentuje dwie główne klasy — **Master** oraz **Slave** — wraz z typami pomocniczymi definiującymi protokół komunikacyjny.



Rysunek 3.1. Diagram klas biblioteki ModuleComms — relacja Master-Slave z protokołem komunikacyjnym (opracowanie własne)

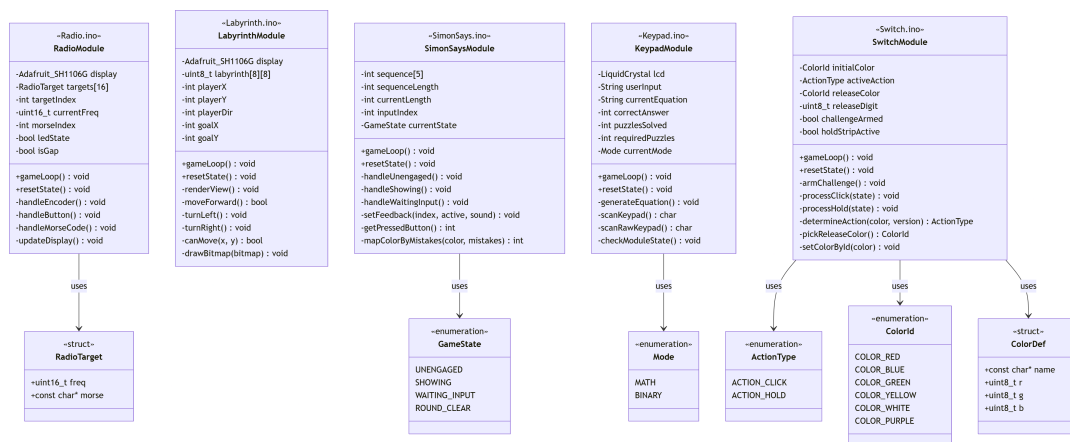
Diagram klas przedstawia strukturę autorskiej biblioteki komunikacyjnej, w tym klasy **Master** (odpowiadającą za jednostkę nadrzędną) oraz **Slave** (reprezentującą podległe moduły), a także powiązane z nimi typy wyliczeniowe **Protocol** i **Status**. Implementacja wykorzystuje mechanizm wstrzykiwania funkcji zwrotnych (wskaźniki **GameLoop** i **ResetState**), co pozwala na pełną separację warstwy sieciowej od logiki biznesowej konkretnych zadań. Metoda **getModuleStatus()** umożliwia odczyt stanu poszczególnych węzłów.

Klasa **Slave** reprezentuje stronę odbiorczą protokołu. Wykorzystuje statyczne pola i metody ze względu na ograniczenia biblioteki **Wire.h**, która wymaga rejestracji funkcji zwrotnych w postaci wskaźników na funkcje wolnostojące. Wstrzykiwanie logiki gry odbywa się przez wskaźniki funkcyjne **GameLoop** oraz **ResetState**, przekazywane w konstruktorze — stanowi to realizację wzorca odwrócenia sterowania (ang. *Inversion of Control*), umożliwiającą każdemu modułowi zagadki dostarczenie własnej implementacji pętli gry bez modyfikacji kodu biblioteki komunikacyjnej.

Typy wyliczeniowe **Protocol** oraz **Status** definiują zestaw jednobajtowych kodów operacyjnych (*opcodes*) przesyłanych na magistrali I²C, zapewniając jednoznaczność interpretacji ramek komunikacyjnych po obu stronach łącza.

3.4.2 Diagram klas modułów zagadek

Każdy moduł zagadki w systemie stanowi niezależną jednostkę logiczną, integrowaną z magistralą komunikacyjną poprzez instancję klasy **Slave** z biblioteki **ModuleComms**. Diagram przedstawiony na Rysunku 3.2 ilustruje strukturę pięciu zaimplementowanych modułów wraz z ich wewnętrznymi typami danych.



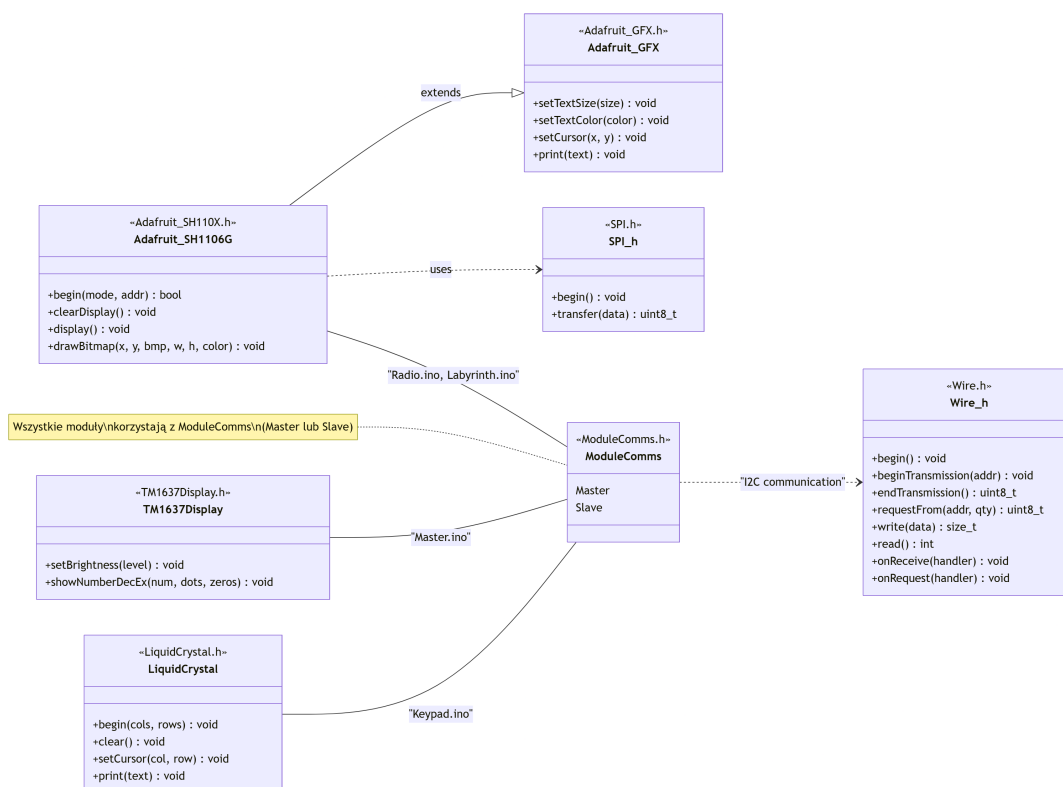
Rysunek 3.2. Diagram klas modułów zagadek z wewnętrznymi typami wyliczeniowymi i strukturami danych (opracowanie własne)

Wspólnym elementem architektury wszystkich modułów jest mechanizm wstrzykiwania funkcji zwrotnych (ang. *callbacks*) — każdy moduł przekazuje w konstruktorze klasy `Slave` wskaźniki na swoje funkcje `gameLoop()` oraz `resetState()`, realizując wzorzec odwrócenia sterowania (ang. *Inversion of Control*). Dzięki temu klasa `Slave` wywołuje logikę specyficzną dla danego modułu bez znajomości jej implementacji.

Moduły różnią się znacząco pod względem złożoności wewnętrznej. Moduł sekwencyjny wykorzystuje maszynę stanów (enum `GameState`) do zarządzania fazami rozgrywki, natomiast moduł z przełącznikiem operuje na typach wyliczeniowych `ActionType` i `ColorId` definiujących logikę decyzyjną opartą na kolorze diody RGB i numerze wersji gry. Moduł arytmetyczny implementuje dwa tryby pracy (enum `Mode`: arytmetyczny i binarny), a moduł radiowy przechowuje tablicę struktur `RadioTarget` z częstotliwościami i kodami Morse’a w pamięci Flash (`PROGMEM`).

3.4.3 Diagram klas warstwy sprzętowej i bibliotek zewnętrznych

Ostatni z diagramów (Rysunek 3.3) przedstawia mapę zależności pomiędzy modułami systemu a wykorzystywanymi bibliotekami zewnętrznymi. Diagram uwidacznia, w jaki sposób poszczególne węzły korzystają ze współdzielonych komponentów programowych.



Rysunek 3.3. Diagram zależności warstwy sprzętowej — powiązania modułów z bibliotekami zewnętrznymi (opracowanie własne)

Centralnym elementem jest biblioteka `ModuleComms`, z której korzystają wszystkie węzły systemu — zarówno Master (klasa `Master`), jak i moduły zagadek (klasa `Slave`). Komunikacja realizowana jest za pośrednictwem biblioteki `Wire.h`, stanowiącej warstwę abstrakcji nad sprzętowym kontrolerem I²C.

Moduły radiowy oraz labiryntu wykorzystują dodatkowo sterownik wyświetlaczy OLED `Adafruit_SH1106G`, który dziedziczy po uniwersalnym rdzeniu graficznym `Adafruit_GFX` i komunikuje się z wyświetlaczem przez interfejs SPI (`SPI.h`). Moduł arytmetyczny korzysta z biblioteki `LiquidCrystal` do obsługi panelu alfanumerycznego LCD, natomiast jednostka nadrzędna wykorzystuje `TM1637Display` do sterowania wyświetlaczem 7-segmentowym prezentującym czas rozgrywki. Moduły sekwencyjny oraz moduł z przełącznikiem nie wymagają zewnętrznych bibliotek peryferiów — operują wyłącznie na pinach cyfrowych mikrokontrolera za pomocą standardowych funkcji Arduino.

4 Implementacja systemu

W niniejszym rozdziale opisano praktyczną realizację założeń projektowych systemu. Nacisk położono na architekturę oprogramowania, implementację autorskiego protokołu komunikacyjnego, zastosowany wzorzec wielozadaniowości kooperatywnej oraz szczegółową budowę poszczególnych modułów zagadek. Dla każdego komponentu przedstawiono kluczowe fragmenty kodu źródłowego wraz z uzasadnieniem przyjętych rozwiązań programistycznych.

4.1 Architektura oprogramowania

Zaprojektowany system działa w architekturze silnie rozproszonej ze scentralizowaną jednostką nadrzędną. Jeden koordynujący serwer (*Master*) nadzoruje liczne moduły podrzędne (*Slave*), a warstwa oprogramowania ujednolica sposób wymiany informacji o stanie poszczególnych węzłów. Podczas projektowania struktury programu przyjęto założenie, że logika centralna ma być odseparowana od implementacji konkretnych modułów i ich czujników oraz elementów wykonawczych.

4.2 Autorska biblioteka komunikacyjna ModuleComms

Biblioteka `ModuleComms` powstała, aby ujednolicić obsługę komunikacji I²C i ograniczyć powielanie kodu. Bez niej każdy moduł musiałby samodzielnie zarządzać buforowaniem danych i dekodowaniem ramek protokołu. Biblioteka przejmuje te zadania, udostępniając funkcje do sprawdzania stanu modułów i obsługi logiki gry.

W protokole komunikacyjnym każde polecenie i każdy status to pojedynczy bajt. Komendy wywołujące akcję po stronie serwera to np. `CMD_START_GAME` czy `CMD_IDENTIFY`, a flagi stanu modułu to `STATUS_UNSOLVED`, `STATUS_PASSED` i `STATUS_FAILED`. Dzięki temu zamiast przysyłać przez magistralę pełne tekstowe identyfikatory poleceń wystarczy pojedynczy bajt — co zmniejsza liczbę przesyłanych danych i zwiększa przepustowość komunikacji. Listing 4.1 prezentuje komplet kodów komend i statusów używanych przez warstwę transportową biblioteki.

```

1  #define CMD_START_GAME          0x01
2  #define CMD_END_GAME            0x02
3  #define CMD_GET_STATUS          0x03
4  #define CMD_SET_STATUS          0x04
5  #define CMD_GET_VERSION         0x05
6  #define CMD_SET_VERSION         0x06
7  #define CMD_IDENTIFY            0x07
8  #define CMD_SET_REMAINING_SECONDS 0x08
9  #define CMD_SET_MISTAKE_COUNT    0x09
10 #define CMD_UNKNOWN             0xFF
11
12 #define STATUS_UNSOLVED 0x01
13 #define STATUS_PASSED   0x02
14 #define STATUS_FAILED   0x03

```

Listing 4.1. Definicje protokołu transportowego I²C w bibliotece ModuleComms

Biblioteka udostępnia dwie główne klasy: **Master** — dla jednostki nadrzędnej — oraz **Slave** — dla modułów podrzędnych.

Klasa **Master** odpowiada za koordynację systemu: skanuje magistralę w poszukiwaniu modułów (`discoverModules()`), przesyła pozostały czas gry (`sendRemainingSeconds()`) oraz rozgłasza liczbę błędów (`broadcastMistakeCount()`). Wykaz adresów modułów przechowywany jest w tablicy, co eliminuje potrzebę dynamicznej alokacji pamięci.

Listing 4.2 pokazuje publiczny interfejs klasy **Master**, odpowiedzialnej za orkiestrację rozgrywki.

```

1  class Master {
2  public:
3      Master(uint8_t version);
4      void begin();
5      void discoverModules();
6      void startGame();
7      void endGame();
8      void restartFailedModule(uint8_t index);
9
10     // Obieg stanu i propagacja zmiennych gry do wbudowanych
11     zagadek
12     uint8_t getModuleStatus(uint8_t index);
13     void sendRemainingSeconds(uint8_t moduleAddress, uint16_t
14         remainingSeconds);
15     void broadcastMistakeCount(uint8_t mistakeCount);
16     void sendCommand(uint8_t moduleAddress, uint8_t command);
17 private:
18     uint8_t version;
19     uint8_t moduleAddresses[11];
20     uint8_t moduleCount;
21 };

```

Listing 4.2. Interfejs główny platformy nadzorującej (ModuleComms.h)

Dzięki takiej architekturze **Master** nie musi znać szczegółów działania poszczególnych modułów. Każdy z nich jest traktowany jako „czarna skrzynka”, która zwraca jedynie informację o swoim stanie: sukces, błąd lub oczekiwanie.

Integracja modułu z biblioteką polega na przekazaniu dwóch funkcji: **gameLoopFn** (logika zagadki) oraz **resetStateFn** (powrót do stanu początkowego). Jest to realizacja wzorca odwrócenia sterowania (ang. *Inversion of Control*) — moduł dostarcza kod, a biblioteka decyduje o momencie jego wywołania. Programista musi jedynie wywołać **slave.slaveLoop()** w głównej pętli Arduino.

Listing 4.3 zawiera uproszczony interfejs klasy **Slave** wraz z funkcjami zwrotnymi.

```

1 typedef void (*GameLoop)();
2 typedef void (*ResetState)();
3
4 class Slave {
5 public:
6     Slave(uint8_t address, GameLoop loop, ResetState setState,
7           uint8_t pin);
8
9     // Obsługa niewidocznych funkcji wejścia:
10    void slaveLoop();
11
12    // Publiczne emitery aktualizujące stan logiczny flag na
13    // maszynie stanów
14    void pass();
15    void fail();
16
17    // Aktualizacje rozproszone przetrzymujące pozycję z serwera
18    uint8_t getVersion();
19    uint16_t getRemainingSeconds();
20    uint8_t getMistakeCount();
21 private:
22    static GameLoop gameLoop;
23    static ResetState resetState;
24    // Niskopoziomowy dostęp odczytu sprzętowego z podpięcia \
25    // texttt{Wire.h}
26    static void receiveEvent(int numBytes);
27    static void requestEvent();
28 };

```

Listing 4.3. Uproszczona definicja interfejsu polimorficznego do węzłów łamigłówek (ModuleComms.h)

Jednostka Master zarządza komunikacją poprzez cykliczne odpytywanie wszystkich modułów podłączonych do magistrali. Wykorzystuje do tego funkcję `Wire.requestFrom()`, która umożliwia odczyt statusu z każdego węzła. Jeżeli moduł zgłosi stan `STATUS_PASSED`, Master przestaje go odpytywać, uznając zadanie za wykonane. W przypadku zgłoszenia `STATUS_FAILED` licznik błędów systemu zostaje zwiększony. Po przekroczeniu dopuszczalnej liczby błędów lub upływie czasu Master wysyła do wszystkich modułów sygnał zakończenia gry, co powoduje zablokowanie

ich logiki i wywołanie funkcji resetującej stan urządzenia.

4.3 Wielozadaniowość kooperatywna i wstrzymywania nieblokujące

Standardowe podejście w środowisku Arduino do odmierzania czasu opiera się na funkcji `delay()`. Skutkuje ona jednak zatrzymaniem pracy procesora na określony czas (ang. *busy waiting*), co uniemożliwia wykonywanie innych zadań w tle. Ze względu na asynchroniczny model komunikacji I²C stosowany w niniejszej pracy, użycie blokującego oczekiwania doprowadziłoby do utraty nadchodzących pakietów danych i żądań ze sterownika Master, co skutkowałoby błędami desynchronizacji (ang. *timeout*).

Aby zagwarantować pełną responsywność systemu i uniknąć kolizji na magistrali, wdrożono wzorzec **wielozadaniowości kooperatywnej** (ang. *cooperative multitasking*). Zamiast zatrzymywać program, wykorzystano nieblokujące liczniki programowe (ang. *non-blocking software timers*) oparte na funkcji `millis()`. We wszystkich modułach główna pętla `loop()` wykonuje się nieustannie, a logika programu jedynie sprawdza, czy od ostatniego zdarzenia upłynął wymagany interwał czasowy. Pozwala to mikrokontrolerowi na jednoczesne odświeżanie wyświetlaczy, skanowanie przycisków oraz ciągłe nasłuchiwanie na magistrali I²C bez ryzyka zablokowania komunikacji.

4.4 Implementacja obwodów i modułów wbudowanych

4.4.1 Adresacja i topologia modułów

W implementacji systemu każdy moduł otrzymał stały adres magistrali I²C, co pozwala na jednoznaczną identyfikację urządzeń już na etapie inicjalizacji oraz eliminuje ryzyko kolizji adresów. Rozwiązanie to upraszcza procedurę wykrywania, ułatwia diagnostykę i zwiększa rozszerzalność systemu, ponieważ dodanie nowego modułu sprowadza się do przydzielenia mu wolnego adresu i nie wymaga dodatkowej ingerencji po stronie modułu nadrzędnego. Taka forma adresacji jest również korzystna z punktu widzenia czytelności kodu i organizacji całej topologii logicznej.

Przyjęto następujące przypisanie adresów do modułów, zestawione w Tabeli 4.1:

Tabela 4.1. Przypisanie adresów I²C do modułów systemu

Adres	Moduł
0x11	Moduł sekwencyjny
0x12	Moduł labiryntu
0x13	Moduł radiowy
0x14	Moduł z przewodami
0x15	Moduł symboli

4.4.2 Moduł nadrzędny

Stacja nadrzędna stanowi centralny punkt decyzyjny całego omawianego systemu. Jako jedyna z peryferiów, nie implementuje ona podrzędnego interfejsu zagadek, lecz służy za główny zegar odliczający i dystrybutor globalnych ustawień gry. Stacja nadrzędna została rozbudowana o poczwórny wyświetlacz 7-segmentowy (TM1637), pełniący rolę głównego wskaźnika czasu, oraz dodatkowy, 2-cyfrowy wyświetlacz 7-segmentowy obsługiwany przez rejestry przesuwne. Ten drugi służy do prezentacji wylosowanego ziarna pseudolosowości (zmiennej `version`), determinującego logikę pozostałych modułów. Interfejs wyposażono również w przycisk Start, przełącznik dźwigniowy oraz przetwornik piezoelektryczny (buzzer) do sygnalizacji dźwiękowej.

Mikrokontroler reguluje tempo narastających ostrzeżeń dźwiękowych. Stabilność tego modułu opiera się na nieblokującej funkcji `handleBeeps()`, której algorytm wyznacza odstępy między sygnałami w oparciu o różnicę między pozostałym czasem gry a progiem krytycznym (`steadyBeepThreshold`). Gdy rezerwa czasu spadnie poniżej progu, sygnał staje się ciągły. Wykorzystano do tego wartość zmiennoprzecinkową `progress`, która estymuje wymagany sukcesywny spadek interwału `beepInterval`. Dzięki rezygnacji z instrukcji `delay()`, Master zachowuje pełną responsywność na pakiety przychodzące z magistrali I²C.

Listing 4.4 przedstawia implementację funkcji `handleBeeps()`, która dynamicznie skraca interwał sygnałów ostrzegawczych.

```

1 void handleBeeps() {
2     if (millis() - lastBeepTime >= beepInterval) {
3         beep();
4         lastBeepTime = millis();
5
6         // Kalkulacja przyspieszania odstępów w miarę upływu czasu
7         if (remainingTime > steadyBeepThreshold) {
8             float progress = float(remainingTime - steadyBeepThreshold)
9                             / float(gameDurationSeconds -
10                                steadyBeepThreshold);
11             beepInterval = finalInterval + (initialInterval -
12                finalInterval) * progress;
13         } else {
14             beepInterval = finalInterval; // Sygnał ciągły po
15             przekroczeniu progu
16         }
17     }
18 }

```

Listing 4.4. Nieblokująca obsługa przyspieszonego upływu czasu ostrzegawczego (Master.ino)

Ważnym elementem zabezpieczającym obsługę przełącznika jest funkcja `handleSwitch()`. Aby zapobiec przypadkowemu przerwaniu rozgrywki, wdrożono mechanizm potwierdzenia resetu. Operator (wyłącznie w celach diagnostycznych) musi trzykrotnie przełączyć dźwignię w krótkim odstępie czasu, zdefiniowanym przez `SWITCH_RESET_TIMEOUT`, aby wywołać procedurę `resetGame()` i przywrócić system do stanu gotowości.

Listing 4.5 ilustruje licznik okna czasowego używany do bezpiecznego potwierdzenia resetu.


```

1 void handleSwitch() {
2     bool currentSwitchState = (digitalRead(SWITCH_PIN) == LOW);
3     if (currentSwitchState != isSwitchOn) {
4         isSwitchOn = currentSwitchState; // Detekcja przerzucenia prądu
5                                         // na przełączniku
6         if (!isSwitchOn) {
7             unsigned long currentTime = millis();
8
9             // Zliczanie prób na oknie resetów
10            if (currentTime - lastSwitchOffTime < SWITCH_RESET_TIMEOUT) {
11                switchOffCount++;
12                if (switchOffCount >= 3) {
13                    resetGame(); // Resetowanie gry i powiadomienie modułów
14                                // Slave
15                    switchOffCount = 0;
16                }
17            } else {
18                switchOffCount = 1;
19            }
20            lastSwitchOffTime = currentTime;
21        } else {
22            if (!isGameInProgress) startGame();
23        }
24    }
25 }

```

Listing 4.5. Programowe ubezpieczenie przed przypadkowym restartem gry z licznikiem czasowym operacji (Master.ino)

Kluczowa dla przebiegu gry jest funkcja `checkModules()`, która odpowiada za sprawdzenie warunków zwycięstwa lub porażki. Master sekwencyjnie pobiera statusy ze wszystkich zarejestrowanych modułów (`STATUS_PASSED`, `STATUS_FAILED` lub `STATUS_UNSOLVED`). Na tej podstawie system decyduje, czy cała zagadka została rozwiązana, czy też należy zarejestrować błąd popełniony przez graczy.

Listing 4.6 pokazuje pętlę odpytywania modułów i priorytetowe przechwytywanie stanu błędu.

```

1 void checkModules() {
2     areAllModulesSolved = true;
3     failedModuleIndex = -1;
4     for (uint8_t i = 0; i < master.getModuleCount(); i++) {
5         uint8_t status = master.getModuleStatus(i);
6         // Jedyna definicja sukcesu dla całego urządzenia: wszystkie zg
           łaszają PASS
7         if (status != STATUS_PASSED) {
8             areAllModulesSolved = false;
9         }
10        // Przechwytywanie błędu jako absolutny priorytet iteracji
11        if (status == STATUS_FAILED) {
12            failedModuleIndex = i;
13            break;
14        }
15    }
16 }

```

Listing 4.6. Procedura odpytywania stacji na nasłuchu I²C (Master.ino)

Główna logika sterownika Master znajduje się w pętli `loop()`. Gdy gra trwa (`isGameInProgress`), system zarządza czterema kluczowymi zadaniami: odliczaniem czasu (`updateGameTimer()`), sygnalizacją dźwiękową (`handleBeeps()`), obsługą diody błędu (`handleMistakeBlinking()`) oraz cyklicznym sprawdzaniem statusu podłączonych urządzeń. Funkcja `handleMistakeBlinking()` informuje o liczbie błędów poprzez serię błysnięć czerwonej diody LED powtarzaną co 5 sekund. W przypadku wykrycia awarii modułu, Master uruchamia procedurę `handleFailure()`, która sygnalizuje błąd dźwiękiem i resetuje dany moduł, pozwalając na ponowne rozwiązanie zagadki.

Listing 4.7 przedstawia kompletną pętlę główną jednostki nadrzędnej z podziałem na zadania cykliczne.

```

1 void loop() {
2     handleSwitch();    // Odczyt statusu zasilania/restartu
3     handleButton();    // Manualne uruchomienie po starcie
4
5     if (isGameEnded && !isSwitchOn) {
6         resetGame();
7         isGameEnded = false;
8     }
9
10    if (isGameInProgress) {
11        updateGameTimer();
12        handleBeeps();
13        handleMistakeBlinking();
14
15        if (isHandlingFailure) {
16            handleFailure(); // Procedura pauzująca wymuszona błędnym
                             // stanem modułu
17        } else {
18            checkModules(); // Aktualizacja stanu wszystkich modułów
19            checkWinLose(); // Weryfikacja warunków końca gry
20        }
21
22        delay(50); // Bufor mitygujący zapchanie magistrali I2C
23    }
24 }

```

Listing 4.7. Główna pętla sterująca jednostki nadrzędnej (Master.ino)

4.4.3 Moduł sekwencyjny

Implementuje klasyczną łamigłówkę typu *Simon Mówi*, w której gracz musi zapamiętać i bezbłędnie odwzorować rosnącą sekwencję sygnałów świetlno-dźwiękowych. Warstwa sprzętowa modułu opiera się na czterech przyciskach mechanicznych przypisanych do kolorów oraz przetworniku piezoelektrycznym. Przebieg rozgrywki podzielono na fazy prezentacji (SHOWING) oraz oczekiwania na wejście (WAITING_INPUT), zarządzane przez maszynę stanów.

Kluczowym mechanizmem utrudniającym zadanie jest dynamiczne przypisywanie kolorów przycisków w zależności od aktualnej liczby błędów popełnionych w całej grze. Funkcja `mapColorByMistakes()` sprawia, że ten sam kolor sygnału mo-

że wymagać naciśnięcia innego przycisku w miarę narastania presji czasu i liczby pomyłek przesyłanych magistralą I²C.

Koordinacja stanów gry odbywa się wewnątrz funkcji `gameLoop()`, która w każdym cyklu procesora sprawdza aktualny etap rozgrywki i wywołuje odpowiednie procedury obsługi prezentacji sekwencji lub odczytu wejść.

Listing 4.8 ilustruje przejścia maszyny stanów pomiędzy fazami rundy modułu sekwencyjnego.

```
1 void gameLoop() {
2     switch (currentState) {
3         case UNENGAGED:    handleUnengaged();    break;
4         case SHOWING:      handleShowing();      break;
5         case WAITING_INPUT: handleWaitingInput(); break;
6         case ROUND_CLEAR:
7             if (millis() - stateTimer >= WAIT_AFTER_CORRECT) {
8                 showIndex = 0;
9                 inputIndex = 0;
10                currentState = SHOWING;
11                stateTimer = millis();
12            }
13            break;
14     }
15 }
```

Listing 4.8. Główna pętla logiczna modułu sekwencyjnego (SimonSays.ino)

Implementacja ta pozwala na zachowanie ciągłej łączności z jednostką Master przy jednoczesnej obsłudze animacji diod oraz sygnałów dźwiękowych.

Listing 4.9 pokazuje reguły mapowania kolorów przycisków zależnie od liczby błędów globalnych.

```

1 int mapColorByMistakes(int shownColor, uint8_t mistakeCount) {
2     if (shownColor < 0 || shownColor > 3) return shownColor;
3
4     if (mistakeCount == 0) {
5         const int map0[4] = {BLUE, RED, GREEN, YELLOW};
6         return map0[shownColor];
7     }
8     if (mistakeCount == 1) {
9         const int map1[4] = {YELLOW, GREEN, BLUE, RED};
10        return map1[shownColor];
11    }
12    const int map2plus[4] = {GREEN, YELLOW, RED, BLUE};
13    return map2plus[shownColor];
14 }

```

Listing 4.9. Dynamiczne przypisywanie układu klawiszy w zależności od liczby błędów (SimonSays.ino)

4.4.4 Moduł arytmetyczny

Realizuje łamigłówkę matematyczną, w której zadaniem gracza jest rozwiązanie proceduralnie generowanego zadania i wprowadzenie wyniku za pomocą klawiatury. Warstwa sprzętowa modułu składa się z 16-klawiszowej klawiaturze membranowej w topologii matrycowej oraz alfanumerycznego wyświetlacza LCD 16x2. W przeciwieństwie do założeń wykorzystujących adapter I²C, w ostatecznej implementacji zdecydowano się na bezpośrednie sterowanie równoległe (6 linii sterujących), co pozwoliło na uproszczenie warstwy programowej kosztem wykorzystania większej liczby pinów mikrokontrolera (od D11 do A2).

Główna pętla gry `gameLoop()` zarządza stanem wyświetlacza, czasem oczekiwania po błędzie oraz interpretacją wejść z klawiatury.

Listing 4.10 przedstawia przebieg pętli głównej modułu arytmetycznego wraz z kontrolą pauzy po błędzie.

```

1 void gameLoop() {
2     // Obsługa wymuszonej pauzy po popełnieniu błędu
3     if (waitingAfterMistake) {
4         if (millis() >= nextEquationAt) {
5             waitingAfterMistake = false;
6             generateEquation();
7         } else {
8             return;
9         }
10    }
11    // Odświeżenie interfejsu przy zmianie zadania
12    if (updateDisplay) {
13        lcd.clear();
14        lcdPrintLine(0, (currentMode == BINARY) ? lastBinary :
15                     currentEquation);
16        updateDisplay = false;
17    }
18    // Skanowanie wejść od gracza
19    char key = scanKeypad();
20    if (key != '\0') {
21        handleKey(key); // Rozpoznawanie liczb i znaków funkcyjnych
22    }
23 }

```

Listing 4.10. Główna pętla logiczna modułu arytmetycznego (Keypad.ino)

Algorytm generuje zadania w dwóch trybach (ustalanych na podstawie globalnej zmiennej `version`): arytmetycznym (podstawowe działania matematyczne) oraz binarnym (konwersja liczb dziesiętnych na binarne). Wdrożenie obsługi wejścia wymagało precyzyjnego mechanizmu eliminowania drgań styków (ang. *debouncing*), aby uniknąć rejestrowania wielokrotnych naciśnień tego samego klawisza.

Listing 4.11 zawiera implementację programowego debouncera dla matrycy klawiatury.

```

1 char scanKeypad() {
2     char rawKey = scanRawKeypad(); // Odczyt sprzętowy matrycy
3     unsigned long now = millis();
4
5     // Detekcja zmiany stanu klawisza
6     if (rawKey != lastRawKey) {
7         lastRawKey = rawKey;
8         keyLastChangeAt = now;
9     }
10
11    // Filtrowanie drgań styków (ignorowanie szybkich oscylacji)
12    if (now - keyLastChangeAt < KEYPAD_DEBOUNCE_MS) {
13        return '\0';
14    }
15
16    // Rejestracja stabilnego wciśnięcia
17    if (rawKey != debouncedKey) {
18        debouncedKey = rawKey;
19        if (debouncedKey == '\0') {
20            keyReported = false; // Reset po puszczeniu klawisza
21        }
22    }
23
24    // Jednorazowe zgłoszenie naciśnięcia do logiki gry
25    if (debouncedKey != '\0' && !keyReported) {
26        keyReported = true;
27        return debouncedKey;
28    }
29    return '\0';
30 }

```

Listing 4.11. Nieblokujący odczyt klawiatury z programowym debouncingiem (Keypad.ino)

4.4.5 Moduł z przełącznikiem

Realizuje prostą, lecz wymagającą precyzji zagadkę, w której zadaniem gracza jest wykonanie właściwej akcji na fizycznym przełączniku dźwigniowym w oparciu o aktualny kolor podświetlenia diody RGB. W zależności od koloru i numeru wersji rozgrywki (`version`), moduł oczekuje jednej z dwóch akcji: szybkiego przełączenia

(*Click*) lub przytrzymania przełącznika (*Hold*) do momentu wystąpienia konkretnej cyfry na liczniku głównym.

Koordinację stanów modułu realizuje funkcja `gameLoop()`, która monitoruje fizyczny stan pinu wejściowego i w zależności od wybranego trybu wywołuje procedury przetwarzające czas trwania interakcji.

Listing 4.12 przedstawia logikę wyboru procedury *Click/Hold* w pętli modułu z przełącznikiem.

```
1 void gameLoop() {
2     // Inicjalizacja wyzwania przy starcie lub po resecie
3     if (!challengeArmed) {
4         armChallenge();
5         return;
6     }
7
8     int currentState = digitalRead(SWITCH_PIN);
9
10    // Wybór procedury weryfikacji w zależności od trybu
11    if (activeAction == ACTION_CLICK) {
12        processClick(currentState);
13    } else {
14        processHold(currentState);
15    }
16 }
```

Listing 4.12. Główna pętla logiczna modułu z przełącznikiem (Switch.ino)

W przypadku akcji *Hold*, dioda RGB zmienia kolor w momencie aktywacji przełącznika, podpowiadając graczowi moment zwolnienia dźwigni. Użytkownik musi puścić przełącznik w chwili, gdy ostatnia cyfra odliczanego przez Mastera czasu gry zgadza się z wartością przypisaną do danego koloru (np. cyfra 4 dla koloru niebieskiego).

Listing 4.13 pokazuje reguły decyzyjne przypisujące akcję do kombinacji koloru i wersji rozgrywki.


```

1 ActionType determineAction(ColorId color, uint8_t version) {
2     if (color == COLOR_GREEN) return ACTION_CLICK;
3
4     // Logika warunkowa oparta na parzystości wersji
5     if ((color == COLOR_RED || color == COLOR_BLUE) && (version % 2
6         == 0)) {
7         return ACTION_CLICK;
8     }
9     if ((color == COLOR_BLUE || color == COLOR_YELLOW) && (version
10        % 3 == 0)) {
11        return ACTION_HOLD;
12    }
13    if (color == COLOR_YELLOW && (version % 2 == 0) && (version % 3
14        == 0)) {
15        return ACTION_CLICK;
16    }
17    return ACTION_HOLD; // Domyślnie wymagane przytrzymanie
18 }

```

Listing 4.13. Wyznaczanie poprawnej procedury działania w oparciu o stan gry (Switch.ino)

4.4.6 Moduł labiryntu

Implementuje przestrzenną łamigłówkę nawigacyjną, w której gracz porusza się wewnątrz wirtualnego labiryntu za pomocą trzech przycisków (ruch na wprost oraz obroty w prawo lub w lewo). Interfejs graficzny oparto na wyświetlaczu OLED 128x64 pikseli. Co istotne, w przeciwieństwie do innych modułów, system ten komunikuje się z ekranem za pomocą szybkiej magistrali SPI (piny MOSI, CLK, CS, DC, RST). Wybór tego standardu zamiast I²C podyktowany był nie tylko wyższą przepustowością niezbędną do odświeżania pełnoekranowych bitmap, ale przede wszystkim ograniczeniem sprzętowym mikrokontrolera. ATmega328P posiada tylko jeden interfejs I²C, który w tym przypadku musi stale pracować w trybie Slave, by odbierać komendy od Mastera. Równoczesne pełnienie roli Mastera dla wyświetlacza byłoby na tym układzie praktycznie niemożliwe.

Logika modułu opiera się na współdzieleniu funkcji `canMove()` zarówno do walidacji przemieszczeń gracza, jak i do dynamicznego generowania rzutu perspektywicznego. Sercem koordynacji jest funkcja `gameLoop()`, która w sposób nieblokujący

zarządza stanem wejść i wyjść.

Listing 4.14 przedstawia nieblokującą obsługę wejść użytkownika i renderowania widoku labiryntu.

```

1 void gameLoop() {
2     // Pierwszorzędne narysowanie widoku po starcie gry
3     if (needsInitialRender) {
4         renderView();
5         needsInitialRender = false;
6     }
7     // Odczyt stanów przycisków sterujących
8     bool btnLeft      = digitalRead(BTN_LEFT);
9     bool btnForward   = digitalRead(BTN_FORWARD);
10    bool btnRight      = digitalRead(BTN_RIGHT);
11
12    unsigned long currentMillis = millis();
13
14    // Obsługa wejścia z programowym debouncingiem
15    if (currentMillis - lastInputTime >= INPUT_DELAY) {
16        if (!btnForward && lastBtnForward) {
17            lastInputTime = currentMillis;
18            if (moveForward()) {
19                display.clearDisplay(); // Czyszczenie po wygranej
20                display.display();
21                return;
22            }
23            renderView();
24        }
25        else if (!btnLeft && lastBtnLeft) {
26            lastInputTime = currentMillis;
27            turnLeft();
28            renderView();
29        }
30        else if (!btnRight && lastBtnRight) {
31            lastInputTime = currentMillis;
32            turnRight();
33            renderView();
34        }
35    }
36    // Zapamiętanie stanów do detekcji zbocza
37    lastBtnLeft      = btnLeft;
38    lastBtnForward   = btnForward;
39    lastBtnRight     = btnRight;
40 }

```

Listing 4.14. Główna pętla logiczna modułu labiryntu (Labyrinth.ino)

Warstwa wizualna budowana jest z przeskalowanych bitmap definiujących ściany w różnej odległości i perspektywie. Wszystkie elementy graficzne (np. `epd_bitmap_closed_left_far`) przechowywane są w pamięci Flash za pomocą dyrektywy `PROGMEM`. Każda bitmapa o wymiarach 128x64 piksele zajmuje 1024 bajty (1 KB) pamięci nieulotnej, gdzie pojedynczy bajt definiuje stan ośmiu sąsiednich pikseli w poziomie. Dzięki takiemu podejściu, mimo że zbiór grafik waży aż 15 KB, moduł mieści się w pamięci ATmega328P, rezerwując jedynie niezbędny stały bufor ramki (1 KB) w pamięci operacyjnej SRAM. Moduł zajmuje 28234 bajty (87%) Flash oraz 792 bajty (38%) SRAM.

4.4.7 Moduł radia

Implementuje łąmigłówkę opartą na dekodowaniu alfabetu Morse’a i strojeniu częstotliwości radiowej. Na module prezentowany jest sygnał nawigacyjny w postaci migającej diody LED wysokiej jasności, odwzorowującej rytm kodu Morse’a. Zadaniem gracza jest odsłuchanie i rozkodowanie sekwencji sygnałów w celu ustalenia słowa kluczowego, a następnie odszukanie powiązanej z nim częstotliwości nadajnika w materiałach referencyjnych i wybranie jej za pomocą inkrementalnego enkodera obrotowego. Poprawne dostrojenie transpondera kończy zagadkę.

Koordinację wszystkich systemów modułu — odświeżania ekranu, obsługi enkodera oraz generowania sygnałów świetlnych — realizuje funkcja `gameLoop()`. Jest ona wywoływana asynchronicznie przez bibliotekę komunikacyjną, co pozwala na zachowanie responsywności węzła przy jednoczesnym wykonywaniu logiki gry.

Listing 4.15 pokazuje kolejność wywołań funkcji odpowiedzialnych za logikę modułu radiowego.

```

1 void gameLoop() {
2     // Pierwszorzędne odświeżenie ekranu po inicjalizacji gry
3     if (needsInitialRender) {
4         updateDisplay();
5         needsInitialRender = false;
6     }
7     handleEncoder();    // Obsługa strojenia częstotliwości
8     handleButton();    // Weryfikacja naciśnięcia przycisku
9     zatwierdzenia
10    handleMorseCode();  // Asynchroniczne generowanie kodu Morse'a
    na diodzie LED
}

```

Listing 4.15. Główna pętla logiczna modułu radiowego (Radio.ino)

Zastosowany model programowania opiera się na maszynie stanów i nieblokujących licznikach `millis()`, dzięki czemu animacja diody LED nie wstrzymuje pracy procesora. Kluczowym wyzwaniem implementacyjnym była optymalizacja pamięci SRAM, na którą duży wpływ ma sterownik wyświetlacza OLED (rezerwujący dynamicznie 1024 bajty na bufor ramki). Aby uniknąć przepełnienia stosu, wszystkie definicje słów i częstotliwości zostały przeniesione do nieulotnej pamięci programu.

Listing 4.16 prezentuje strukturę danych `RadioTarget` oraz deklaracje `PROGMEM` użyte do oszczędzania pamięci SRAM.

```

1  const char m_0[] PROGMEM = "... ..-. .-.";
2  // [...]
3  const char m_15[] PROGMEM = "-... .- - .-.";
4
5  struct RadioTarget {
6      uint16_t freq;          // Częstotliwość docelowa nadajnika (np.
7                               3505 dla 3.505 MHz)
8      const char* morse;      // Wskaźnik do ciągu znaków w pamięci Flash
9  };
10
11  const RadioTarget targets[] = {
12      {3505, m_0},            // shell
13      // [...]
14      {3608, m_15}           // beats
15  };

```

Listing 4.16. Deklaracja haseł telegraficznych i częstotliwości w pamięci Flash (Radio.ino)

5 Testy i działanie systemu

Niniejszy rozdział zawiera szczegółową ocenę systemu z dwóch perspektyw: analizy ograniczeń zasobów sprzętowych mikrokontrolerów ATmega328P oraz empirycznej weryfikacji poprawności komunikacji I²C, niezawodności logiki modułów i precyzji odmierzenia czasu. Obydwie części uzupełniają się wzajemnie — pierwsze sekcje pokazują, w jakich warunkach pracy operuje system, podczas gdy testy demonstrują spełnienie wymagań funkcjonalnych bez przekroczenia limitów sprzętowych.

Specyfika mikrokontrolerów AVR wymusza odmienną metodykę testowania niż w przypadku klasycznych komputerów. Podstawową weryfikacją jest pomyślna kompilacja kodu, która sprawdza nie tylko składnię, ale i zajętość pamięci Flash oraz SRAM — raporty kompilatora pozwalają wstępnie upewnić się, że program będzie mieścił się w dostępnych limitach. Dodatkowo, do podglądu komunikatów diagnostycznych w czasie rzeczywistym wykorzystywano płytkę Arduino Uno R4 WiFi jako przekaźnik UART-USB, monitorując stany wewnętrzne (adresy I²C, statusy modułów, pomiary czasu). Uzupełnieniem weryfikacji kompilacyjnej były testy integracyjne przeprowadzone na fizycznym prototypie, obejmujące zarówno walidację wydajności kanału I²C, jak i pomiary rzeczywistego zużycia pamięci w warunkach operacyjnych.

5.1 Analiza wykorzystania zasobów sprzętowych

Wybór 8-bitowego mikrokontrolera ATmega328P jako platformy bazowej narzucił rygorystyczne wymagania dotyczące optymalizacji kodu. Poniższa tabela (Tabela 5.1) zestawia ostateczne wyniki kompilacji dla wszystkich komponentów systemu, obrazując stopień wykorzystania dostępnej pamięci Flash oraz SRAM.

Tabela 5.1. Zestawienie wykorzystania pamięci mikrokontrolerów ATmega328P

Moduł	Flash [B]	Flash [%]	SRAM [B]	SRAM [%]
Moduł nadrzędny	10106	31%	964	47%
Moduł radiowy	15326	47%	1716*	84%
Kontynuacja na następnej stronie				

Tabela 5.1 – kontynuacja z poprzedniej strony

Moduł	Flash [B]	Flash [%]	SRAM [B]	SRAM [%]
Moduł labiryntu	28234	87%	1816*	89%
Moduł sekwencyjny	7812	24%	480	23%
Moduł arytmetyczny	9344	28%	692	33%
Moduł z przełącznikiem	5452	16%	545	26%

* Wartość uwzględniająca 1024 bajty bufora ramki obrazu alokowanego dynamicznie w pamięci SRAM przez bibliotekę graficzną, niewidocznego w statystykach kompilacji.

Największe wyzwanie pod kątem zasobów Flash stanowił moduł labiryntu, co wynika z konieczności przechowywania licznych map bitowych renderowanych w perspektywie 3D. Natomiast w przypadku pamięci SRAM kluczowym ograniczeniem okazała się obecność wyświetlaczy OLED w modułach radiowym oraz labiryntu. Powyższa analiza (Tabela 5.1) uwzględnia fakt, że sterowniki graficzne (np. `Adafruit_SH1106G`) rezerwują w czasie działania programu dodatkowy bufor ramki o rozmiarze 1024 bajtów (128x64 pikseli monochromatycznych). Ponieważ alokacja ta odbywa się dynamicznie wewnątrz klasy sterownika, nie jest ona raportowana przez kompilator jako stałe zużycie pamięci operacyjnej. W rzeczywistości, po doliczeniu tego bufora, moduł labiryntu operuje na granicy stabilności sprzętowej, wykorzystując blisko 90% dostępnej pamięci SRAM (2 KB), co wymusiło rezygnację z jakichkolwiek innych struktur dynamicznych i oparcie logiki wyłącznie na zmiennych lokalnych oraz stałych `PROGMEM`.

Warto również zauważyć, że relatywnie wysokie użycie pamięci Flash w module radiowym (47%) wynika z przechowywania sekwencji alfabetu Morse’a w postaci czytelnych ciągów znaków (np. kropki i kreski jako typ danych `char`). Rozwiązanie to wybrano ze względu na przejrzystość kodu i łatwość wprowadzania zmian. W przypadku konieczności dalszej optymalizacji, zasoby te można by zredukować poprzez zastosowanie kodowania bitowego (bit-packing), gdzie każda kropka i kreska reprezentowana byłaby przez pojedynczy bit, jednak przy obecnej skali projektu nie było to wymagane.

5.2 Weryfikacja systemu poprzez testy i pomiary

Poniższe sekcje prezentują wyniki empirycznej weryfikacji warstwy sprzętowej i komunikacyjnej systemu. Przeprowadzone scenariusze testowe obejmują walidację kluczowych parametrów funkcjonalnych (stabilność protokołów I²C, obsługa błędów,

precyzja timera), a następnie analizę wydajności systemu na podstawie szczegółowych logów z testów operacyjnych.

5.2.1 Wyniki testów sprzętowych

Tabela 5.2. Wyniki testów sprzętowych systemu modularnego

Oznaczenie	n	Opis scenariusza
T01	10	Powtarzalność inicjalizacji systemu oraz automatycznej detekcji węzłów na magistrali I ² C.
T02	10	Weryfikacja mechanizmu obsługi błędów pojedynczego modułu i propagacja stanu do pozostałych węzłów sieci.
T03	5	Weryfikacja warunku zakończenia sesji.
T04	5	Weryfikacja procedury przekroczenia czasu.

Tabela 5.2 zawiera podsumowanie przeprowadzonych scenariuszy testowych oraz krótką charakterystykę każdego testu.

Test T01: Powtarzalność procesu inicjalizacji

Celem testu była weryfikacja stabilności procedury automatycznej adresacji i wykrywania modułów na magistrali I²C. Przeprowadzono 10 cykli zimnego startu systemu, monitorując poprawność identyfikacji wszystkich urządzeń peryferyjnych przez jednostkę centralną (Master). Wszystkie próby przebiegły pomyślnie — każdy moduł udzielił potwierdzenia ACK w odpowiedzi na ramkę `CMD_IDENTIFY`, a po uruchomieniu systemu wszystkie jednostki zgłosiły gotowość do pracy.

Test T02: Obsługa stanów awaryjnych

W ramach testu sprawdzono zdolność jednostki Master do rejestracji i dystrybucji zmiennej stanu błędu (`mistakeCount`) po wystąpieniu zdarzenia `STATUS_FAILED` w dowolnym module. Wszystkie testy przebiegły pomyślnie, a system poprawnie zareagował na symulowane błędy, aktualizując stan globalny i przekazując informację do pozostałych modułów.

Test T03: Weryfikacja warunku zakończenia sesji

Weryfikacji poddano warunek zakończenia sesji, polegający na zgłoszeniu statusu `PASS` przez wszystkie aktywne moduły. W 5 przeprowadzonych próbach system każdorazowo pozostawał w stanie oczekiwania na potwierdzenie ostatniego modułu przed zadeklarowaniem zakończenia sesji.

Test T04: Weryfikacja procedury przekroczenia czasu

Badanie miało na celu zweryfikowanie poprawności działania mechanizmu zakończenia sesji oraz ilościowe określenie odchyłen pomiaru czasu. W pięciu przeprowadzonych próbach zaobserwowano odchylenia czasu w zakresie 2–6 s względem czasu referencyjnego.

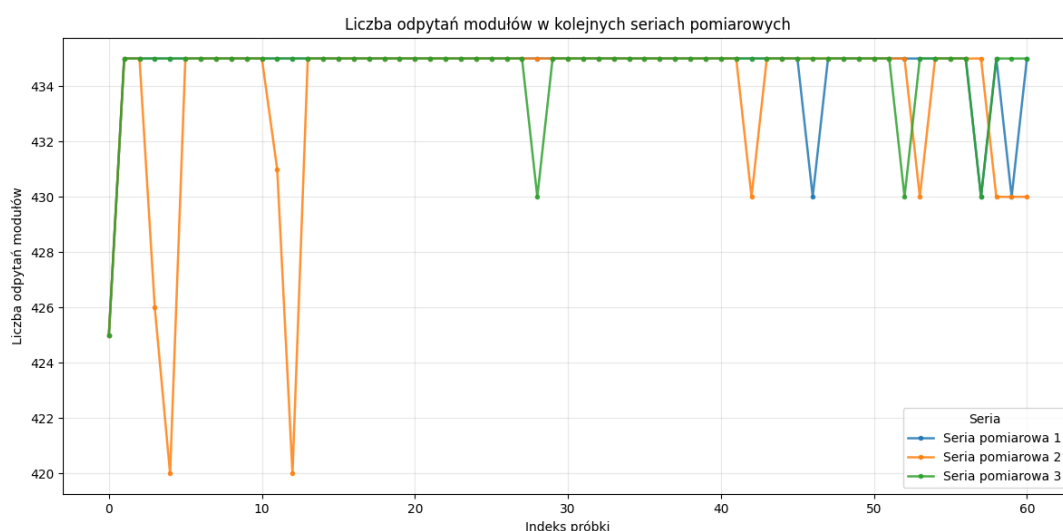
5.2.2 Analiza wydajności i częstotliwości komunikacji

Charakteryzacja pełnego działania systemu wymaga szczegółowej analizy wydajności i intensywności komunikacji I²C. Przeprowadzono analizę danych zbieranych przez system w trakcie 300-sekundowych przebiegów testowych. W ramach trzech niezależnych przebiegów testowych zebrano po 61 pięciosekundowych fragmentów danych w każdym (razem 183), umożliwiając ocenę granularnych metryk dotyczących częstotliwości opytania modułów, czasów odkrywania i ogólnej reaktywności systemu. W Tabeli 5.3 zestawiono kluczowe parametry pomiarów.

Tabela 5.3. Metryki wydajności systemu I²C i pętli głównej

Parametr	Wartość	Jednostka
Średni czas wykrywania modułów	115	ms
Liczba odpytań w 5 sekund	434	zapytań
Średnia częstotliwość odpytań	86,8	Hz (całkowita)
Średnia częstotliwość na moduł	17,36	Hz
Średni okres cyklu opytania na moduł	57,6	ms

Przebieg liczby odpytań w kolejnych seriach pomiarowych pokazany na Rysunku 5.1 potwierdza, że system utrzymuje stałą wartość około 435 odpytań na próbkę (5 sekund) z okazjonalnymi wahaniami do 420, co odzwierciedla stabilność mechanizmu opytania magistrali I²C.



Rysunek 5.1. Przebieg liczby odpytań modułów w kolejnych seriach pomiarowych

Pomiary wykazują, że system utrzymuje stałą częstotliwość odpytania każdego z modułów na poziomie 17 Hz, co odpowiada okresowi cyklu około 57–58 ms. Czas odkrywania modułów na magistrali I²C wynoszący średnio 115 ms jest akceptowalny dla zastosowania o charakterze interaktywnym, gdzie elementy interfejsu aktualizują się co kilkaset milisekund. Wynik ten potwierdza efektywność implementacji procedury `discoverModules()` w kodzie modułu nadrzędnego.

6 Podsumowanie i wnioski końcowe

Projekt systemu i przeprowadzone testy umożliwiły pomyślną realizację głównego celu pracy, jakim było zaprojektowanie i wdrożenie modularnego systemu zadań logicznych opartego na mikrokontrolerach ATmega328P. Opracowane rozwiązanie obejmuje jednostkę nadrzędną, pięć niezależnych modułów oraz warstwę komunikacyjną opartą na magistrali I²C. Uzyskane wyniki potwierdziły poprawność przyjętej architektury, stabilność działania logiki modułów oraz skuteczność automatycznej detekcji i synchronizacji urządzeń w ramach jednej, spójnej topologii.

Warto podkreślić, że system został zweryfikowany w ściśle określonych scenariuszach testowych i w granicach możliwości sprzętowych platformy ATmega328P. Największe wyzwania dotyczyły stabilności komunikacji na magistrali I²C oraz ograniczeń pamięci SRAM, szczególnie w przypadku modułu labiryntu. Dodatkowym ograniczeniem była trudność w przeprowadzeniu prostych, pełnych symulacji całego układu, dlatego kluczowe znaczenie miały testy wykonywane na rzeczywistym prototypie. Wymusiło to konsekwentną optymalizację kodu, ograniczenie wykorzystania struktur dynamicznych oraz szerokie zastosowanie pamięci Flash, jednak przyjęte rozwiązania okazały się wystarczające do zapewnienia poprawnej i responsywnej pracy całego systemu.

Wyniki badań przedstawione w rozdziale 5 potwierdzają wysoką powtarzalność i poprawność działania systemu. W testach inicjalizacji (T01) każdorazowo uzyskiwano poprawną detekcję modułów oraz potwierdzenia ACK na magistrali I²C, co potwierdza niezawodność procedury startowej. Test obsługi stanów awaryjnych (T02) wykazał poprawną propagację informacji o błędzie i spójne aktualizowanie stanu globalnego, a scenariusz zakończenia sesji (T03) potwierdził, że logika nadrzędna prawidłowo oczekuje na spełnienie kompletnego warunku ukończenia gry.

Istotnym wnioskiem praktycznym jest również stabilność czasowa i komunikacyjna układu. Pomiary wydajności wykazały średni czas wykrywania modułów na poziomie 115 ms oraz utrzymywanie częstotliwości odpytywania około 17 Hz na moduł, co zapewnia płynną i przewidywalną reakcję systemu. Jednocześnie odchylenia pomiaru czasu obserwowane w teście przekroczenia limitu czasowego (T04) pozostawały w ograniczonym zakresie 2-6 s, co przy zastosowanej klasie sprzętu i charakterze interaktywnej rozgrywki należy uznać za wynik akceptowalny.

W przyszłości system mógłby zostać rozbudowany o kolejne moduły oraz o dodatkowe mechanizmy ułatwiające konfigurację i nadzór nad rozgrywką. Cennym kierunkiem dalszych prac byłyby również aktualizacja i powtórzenie badań po dłuższym

czasie eksploatacji, aby ocenić trwałość przyjętych rozwiązań oraz ich podatność na dalszą rozbudowę. Możliwe byłoby także przeniesienie części funkcji na wydajniejszą platformę, na przykład mikrokontroler ESP32, a w szerszej perspektywie uzupełnienie systemu o komunikację bezprzewodową, zdalną administrację lub nowe moduły wykorzystujące bardziej zaawansowane mechanizmy interakcji.

Bibliografia

- [1] Microchip Technology Inc., *ATmega328P 8-bit AVR Microcontroller Datasheet*, Specyfikacja techniczna (Datasheet), https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.
- [2] Arduino, *Arduino Language Reference (C++)*, Dokumentacja języka i struktury programowania środowiska Arduino IDE, opartego o C/C++, <https://www.arduino.cc/reference/en/>.
- [3] Arduino, *LiquidCrystal Library - API Documentation*, Biblioteka referencyjna obsługi wyświetlaczy alfanumerycznych LCD (standard HD44780), <https://www.arduino.cc/reference/en/libraries/liquidcrystal/>.
- [4] Arduino, *Wire Library - API Documentation*, Biblioteka referencyjna rdzenia sprzętowego komunikacji TWI/I²C interfejsów standardu Arduino, <https://www.arduino.cc/reference/en/language/functions/communication/wire/>.
- [5] Limor Fried, Adafruit Industries, *Adafruit SH110X Library*, Wersja 2.1.8, Zoptymalizowany dla modułów macierzy węzłów, https://github.com/adafruit/Adafruit_SH110x.
- [6] Adafruit Industries, *Adafruit GFX Library*, Rdzeń procedur rysowania struktur 2D w środowisku mikrokontrolerów dla ekranów LCD/OLED, <https://github.com/adafruit/Adafruit-GFX-Library>.
- [7] Avishay Orpaz, *TM1637Display*, Driver wyświetlaczy podłączanych po magistrali 2-wire, <https://github.com/avishorp/TM1637>.
- [8] Stroustrup B., *Język C++. Kompendium wiedzy*, wyd. IV, Helion, Gliwice 2014, ISBN: 978-83-246-8770-1.
- [9] Mahmutbegović A., *C++ w systemach wbudowanych. Skuteczna migracja z C do nowoczesnego C++*, Helion, Gliwice 2025.
- [10] Bancila M., *Nowoczesny C++. Zbiór praktycznych zadań dla przyszłych ekspertów*, Helion, Gliwice 2020, ISBN: 978-83-283-6311-3.
- [11] Josuttis N. M., *C++ Biblioteka standardowa. Podręcznik programisty*, wyd. II, Helion, Gliwice 2014.

- [12] Gamma E., Helm R., Johnson R., Vlissides J., *Wzorce projektowe. Elementy oprogramowania obiektowego wielokrotnego użytku*, WNT, Warszawa 2010.
- [13] Roy P., *Efektywne zarządzanie pamięcią w C++*. Praktyczne strategie i techniki tworzenia lekkiego, bezpiecznego i niezawodnego oprogramowania, Helion, Gliwice 2024, ISBN: 978-83-289-1383-7.
- [14] Blum J., *Odkrywanie Arduino. Narzędzia i techniki inżynierii pełnej czaru*, wyd. II, Helion, Gliwice 2020, ISBN: 978-83-283-6923-8.
- [15] Kwaśniewski S., *Mikrokontrolery AVR w praktyce*, Wydawnictwo BTC, Warszawa 2011.

Spis rysunków

3.1	Diagram klas biblioteki ModuleComms — relacja Master-Slave z protokołem komunikacyjnym (opracowanie własne)	22
3.2	Diagram klas modułów zagadek z wewnętrznymi typami wyliczeniowymi i strukturami danych (opracowanie własne)	23
3.3	Diagram zależności warstwy sprzętowej — powiązania modułów z bibliotekami zewnętrznymi (opracowanie własne)	25
5.1	Przebieg liczby odpytań modułów w kolejnych seriach pomiarowych .	51

Spis tabel

3.1	Wymagania funkcjonalne systemu	17
3.2	Wymagania нефункционалне systemu	19
4.1	Przypisanie adresów I ² C do modułów systemu	31
5.1	Zestawienie wykorzystania pamięci mikrokontrolerów ATmega328P .	47
5.2	Wyniki testów sprzętowych systemu modularnego	49
5.3	Метрики wydajności systemu I ² C i pętli głównej	50